Bio Track

A study guide to the code

A file-by-file, concept-by-concept walkthrough of the agricultural intelligence dashboard, written for a team learning HTML, CSS and JavaScript on a real codebase.

30 source files · about 8,000 lines · 5 pages · 6 services · 2 live APIs

Generated 10 October 2026 from the code as it stands

MARKET BOARD · CROPS · ₦/KG · MAIZE ₦880.95 · RICE (PADDY) ₦1,074.07 · CASSAVA ₦562.71

Read it with the code open. Every section names the file and the function it is describing.

Contents

Contents 2

How to use this guide 4

Running the project . . . 4

Part 1 — The big picture 6

1.1 What the software is for . . . 6

1.2 The five pages . . . 6

1.3 How the files fit together . . . 6

1.4 The render loop in one paragraph . . . 7

Part 2 — Patterns you will meet in every file 8

2.1 The namespace and the wrapper function . . . 8

2.2 var, function scope, and why not let/const . . . 8

2.3 data-el: how JavaScript finds elements . . . 8

2.4 Build once, update in place . . . 9

2.5 Publish and subscribe . . . 9

2.6 Status objects . . . 10

2.7 fetch, Promises and timeouts . . . 10

2.8 Storage that cannot break the page . . . 10

2.9 Relative dates in the data . . . 11

2.10 The severity vocabulary . . . 11

Part 3 — File by file 12

3.1 js/config.js — the data . . . 12

3.2 js/dom.js — DOM helpers . . . 13

3.3 js/store.js — persistence . . . 13

3.4 js/format.js — formatting . . . 14

3.5 js/icons.js — the icon sprite . . . 14

3.6 js/services/market.js — market prices . . . 14

3.7 js/services/weather.js — weather intelligence . . . 16

3.8 js/services/irrigation.js — irrigation control . . . 17

3.9 js/services/pests.js — pest and disease control . . . 17

3.10 js/services/harvest.js — harvest alerts . . . 18

3.11 js/services/livestock.js — livestock . . . 18

3.12 js/analytics.js — inventory analytics . . . 19

3.13 js/charts.js — the chart layer . . . 19

3.14 js/shell.js — the shared chrome . . . 20
3.15 js/app.js — the orchestrator . . . 21
3.16 js/pages/*.js — the page modules . . . 21
3.17 js/landing.js and index.html — the landing page . . . 22
3.18 css/styles.css — the dashboard stylesheet . . . 22
3.19 css/landing.css . . . 24
3.20 The HTML pages . . . 24
3.21 tools/fetch-prices.py and data/prices-snapshot.js . . . 25
3.22 js/services/tasks.js — today's jobs . . . 25

Part 4 — Walkthroughs 27

4.1 Opening cockpit.html . . . 27
4.2 Clicking "Record treatment" on cassava mealybug . . . 27
4.3 Changing the location . . . 28
4.4 Opening a valve . . . 28

Part 5 — The maths, with worked examples 29

5.1 What a lot is worth . . . 29
5.2 Spoilage and value at risk . . . 29
5.3 The price signal . . . 29
5.4 Sell priority . . . 29
5.5 Farm value over time . . . 29
5.6 Irrigation: the FAO-56 water balance . . . 30
5.7 Pests: growing degree days . . . 30
5.8 Diseases: infection windows . . . 30
5.9 Livestock: carcass value and eggs . . . 30
5.10 Heat stress: the temperature–humidity index . . . 30

Part 6 — Glossary 31

JavaScript and browser terms as used here . . . 31
Farming terms . . . 32

Part 7 — A study plan for two 33

Exercises, easiest first . . . 33

Appendix A — Where to change what 34

Appendix B — Debugging checklist 34

How to use this guide

This is not a tutorial on JavaScript. It is a map of **this** codebase: what each file does, how the files talk to each other, why they are shaped the way they are, and the maths behind every number on screen. It assumes you can read a function and follow an **if**; everything else is explained the first time it appears.

It is organised in layers. Read Part 1 straight through — it is short and everything else hangs off it. Read Part 2 before you open any file, because the same eight patterns appear in all of them. Part 3 is the reference: one section per file, in the order the browser loads them. Parts 4 and 5 are where understanding turns into confidence — trace the walkthroughs with the browser DevTools open, and check the worked examples against the numbers on your screen.

Three kinds of box appear throughout:

- **Try this** — something to do in the editor or the browser. Do them; that is the whole point.
- **Why it is built this way** — the reasoning behind a choice you might otherwise change.
- **Watch out** — a mistake that is easy to make in exactly this spot.

Whenever the guide says "open the console", it means the browser DevTools: right-click the page → **Inspect** → the **Console** tab. Errors appear there in red with the file and line number. The **Network** tab shows every request the page makes, and the **Elements** tab shows the live HTML and the CSS rules hitting each element.

> A BUG WORTH REMEMBERING
>
> **How a find-and-replace once broke three pages**
>
> When the brand was renamed from AgroSense to Bio Track, the replace hit the last line of three files — `js/pages/cockpit.js`, `js/pages/farm.js` and `js/pages/live.js` — turning `})(window.AgroSense);` into `})(window.Bio Track);`.
>
> A JavaScript identifier cannot contain a space, so that line was a **syntax error**. The browser refused to run any of the file, the page module never registered itself, and every value on those pages sat at "—". The console said *Uncaught SyntaxError: missing) after argument list* — the clue was there the whole time.
>
> **The fix** was one word on the last line of each of those three files: `window.AgroSense` put back. The brand name shown on screen is a *string* (`'Bio Track'` in shell.js) and can be anything; `AgroSense` in `window.AgroSense` is a *variable name*, and every file must use the same one. A global replace cannot tell the two apart — which is the lesson.
>
> You can catch this kind of thing before opening a browser: `node --check js/pages/cockpit.js` parses a file and prints any syntax error. Running it over every file takes a second.

Running the project

There is no build step and nothing to install. The site is plain HTML, CSS and JavaScript, plus one library (Chart.js) loaded from a CDN. It needs to be served over HTTP rather than opened as a file, because the pages fetch data from two external APIs and read each other's state from browser storage. Any static server will do:

```
python tools/devserver.py          # then open http://localhost:5173/
# or VS Code: install "Live Server", right-click index.html -> Open with Live Server
```

Use `tools/devserver.py` rather than `python -m http.server`. It is the same static server with caching switched off. Plain http.server lets the browser hold on to a `.js` file it has already seen, so you edit a file,

reload, and sit there debugging code that is no longer on disk — a genuinely maddening hour until you spot it. If a change ever seems not to apply, that is the first thing to suspect.

The landing page is `index.html`. “Log in” is a plain link to `cockpit.html` — there is no authentication, by design. The four dashboard pages share a sidebar; the bell in the top bar carries the same alerts to all of them.

Part 1 — The big picture

1.1 What the software is for

Commercial farmers lose money on yesterday's news: they sell into a dip because they did not know the price had moved, or they spray before a storm, or they harvest into a wet week. Bio Track is a **decision cockpit**: it pulls live weather for the farm's own coordinates and real market prices, joins them to what the farm actually holds — lots in store, crops in the ground, animals on the hoof — and turns the join into things to do today.

Everything on screen is one of three joins:

The join	The question it answers	Where it lives
Stock × price	What is the farm worth right now, and what did the last month do to that?	analytics.js, livestock.js
Forecast → action	What should I do, or not do, in the next 72 hours?	weather.js, irrigation.js, pests.js, harvest.js, livestock.js
Rules → constraints	What am I not allowed to sell, and for how long?	pests.js → analytics.js

The look, in one paragraph

A soft neutral workspace: the app is one rounded panel floating on a grey backdrop, with an icon rail down the left and the content scrolling on the right. No borders, no shadows — layering is done with tone alone. One saturated green marks the single most important number on each screen, black marks the price board and the active nav chip, and the reserved status colours mark state. The restraint is the point: when almost everything is grey, the one green tile is unmissable.

1.2 The five pages

File	What it shows	Page module
`index.html`	Landing page. Three honest telemetry chips (live weather, a real WFP price, the next harvest) and a "Log in" link.	js/landing.js (standalone)
`cockpit.html`	The one-glance answer: farm value with a 24-month history, five stat tiles, sell priority, advisories, harvest calendar.	js/pages/cockpit.js
`live.html`	The two external feeds with their provenance: the seven-day forecast charts; market tables, trend charts and the source card.	js/pages/live.js
`operations.html`	Irrigation control (soil water deficit per zone, valves) and pest control (degree days, infection windows, spray window, treatments).	js/pages/operations.js
`farm.html`	Lots in store (sortable), fields with the harvest calendar, and livestock (valuation, heat stress, health calendar).	js/pages/farm.js

1.3 How the files fit together

Think of it as layers. Each layer may only reach *down*. The data file knows nothing about the screen; the services know the data; analytics knows the services; the pages know everything but only draw; and `app.js` decides *when* things happen.

```
cockpit.html  — the page: structure + placeholders (data-shell, data-el)
     │  loads these scripts, in this order
     ▼
┌─ FOUNDATIONS ──────────────────────────────────────────────────────┐
│  config.js      every number the farm can change: crops, lots, zones │
│  store.js       safe localStorage / sessionStorage                 │
│  format.js      ₦, tonnes, dates, "3 days ago"                     │
│  dom.js         el(), setText(), tag(), badge()…                   │
│  icons.js       the SVG sprite                                     │
│  data/prices-snapshot.js   bundled WFP prices (offline fallback)   │
└────────────────────────────────────────────────────────────────────┘
     ▼
┌─ SERVICES  (compute things; know nothing about the screen) ────────┐
│  market.js     prices: WFP via HDX, three tiers, signals           │
│  weather.js    Open-Meteo forecast, normalised + advisories        │
│  irrigation.js FAO-56 water balance + the valve controller         │
│  pests.js      degree days, infection windows, spray window, PHIs  │
│  livestock.js  herd valuation, heat stress (THI), health calendar  │
│  harvest.js    maturity windows joined to forecast, price and PHI  │
│  tasks.js      today's jobs, collapsed out of all of the above     │
└────────────────────────────────────────────────────────────────────┘
     ▼
analytics.js   joins stock to prices: value, margin, spoilage, priority
charts.js      Chart.js wiring; create once, update in place
shell.js       builds sidebar, top bar, price board, alerts, footer
pages/X.js     draws THIS page's sections from a context object
app.js         boots everything; compute() → render on every change
```

WHY IT IS BUILT THIS WAY

Why script order matters

Every file ends by attaching something to one shared object: `NS.fmt = {…}`, `NS.market = {…}`, `NS.pages.cockpit = {…}`. A file can only use what has already been attached, so `config.js` must load before `format.js` (which reads `cfg.locale`), the services before `analytics.js`, and `app.js` last of all, because it calls everything. Look at the `<script>` list at the bottom of any dashboard page: that list *is* the dependency graph.

1.4 The render loop in one paragraph

When anything changes — a price arrives, the forecast loads, a valve opens, a treatment is recorded — `app.js` runs `compute()`. That function asks every service for its current answer and packs the results into one **context object** (`ctx`). Then it hands `ctx` to the shell (for the sidebar counts and the price board) and to the page module (for the sections). The page module never fetches or calculates anything; it reads `ctx` and writes text into elements. This one-way flow — *services → compute → ctx → draw* — is the spine of the whole application, and once you see it, every file makes sense.

Part 2 — Patterns you will meet in every file

These eight ideas cover most of the code. Learn them here once; the file-by-file section then reads quickly.

2.1 The namespace and the wrapper function

Every JavaScript file has the same skeleton:

```
window.AgroSense = window.AgroSense || {};   // only in config.js, the first file

(function (NS) {
  'use strict';

  var cfg = NS.config;          // read what earlier files attached

  function helper() { … }       // private: nobody outside can see this

  NS.fmt = { money: …, mass: … };   // public: attached to the shared object
})(window.AgroSense);
```

- `window.AgroSense = window.AgroSense || {};` — *use the existing object if there is one, otherwise make an empty one*. The `||` is "or": if the left side is undefined it falls through to the right. Only the first file needs this line.
- `(function (NS) { … })(window.AgroSense);` — a function that is defined and immediately called, with the shared object passed in as `NS`. Everything declared inside is invisible from outside, so files cannot accidentally overwrite each other's variables. The pattern is called an **IIFE** (immediately-invoked function expression).
- `'use strict';` — asks the browser to be stricter: undeclared variables become errors instead of silently becoming globals.
- The last line is the one the rename broke. `window.AgroSense` is a *property name on the window*; it has to be the same spelling in all 20 files.

> TRY THIS
>
> **See the shared object**
>
> Open the cockpit, open the console, type `AgroSense` and press Enter. Expand it. Every key — `config`, `fmt`, `market`, `pages` — was attached by one file. Type `AgroSense.market.getQuotes()` to see the live price objects. Type `AgroSense.fmt.money(48215000)` and you get `₦48,215,000`.

2.2 var, function scope, and why not let/const

The code uses `var` and `function` declarations everywhere, and no arrow functions, template strings or classes. This is **ES5** JavaScript, chosen so the dashboard runs unmodified in older browsers on farm laptops and phones. If you add new code you may use `let`, `const` and arrows — modern browsers are fine — but be consistent within a file. The one thing to remember about `var`: it is scoped to the *function*, not the block, so a `var` inside an `if` is visible after the `if`.

2.3 data-el: how JavaScript finds elements

The HTML never uses `id` for things JavaScript writes into. It uses a data attribute:

```
<div class="tile-value" data-el="tileMass">—</div>          <!-- HTML: a placeholder -->
```

```
setText(el('tileMass'), fmt.mass(s.totalKg));                // JS: fill it
```

`el(name)` in `dom.js` is just `document.querySelector('[data-el="' + name + '"]')`. The convention keeps the two halves in sync: search the project for `tileMass` and you find exactly one HTML placeholder and one JS line that fills it. `id` is reserved for things that need to be unique for other reasons — chart canvases, section anchors that the sidebar links to (`#irrigation`), and elements with `aria-controls`.

> WATCH OUT
>
> **A typo in a data-el name fails silently**
>
> `el("tileMas")` returns `null`, and `setText(null, …)` is written to do nothing rather than crash. So a misspelt name shows up as a tile stuck on "—", not as an error. When that happens, search for the name in both the HTML and the JS and compare them character by character.

2.4 Build once, update in place

The dashboard redraws often — every price refresh, every irrigation clock tick. If each redraw rebuilt the HTML from a string, the page would flicker, your text selection would vanish, every chart would be thrown away and recreated, and a button you were about to click would be replaced under your finger. So the code separates **building** from **updating**:

```
function buildLotRows(lots) {        // once: create the <tr>s, keep a handle to each cell
  lots.forEach(function (lot) {
    var tdPrice = tag('td', 'right num');
    …
    lotRows[lot.id] = { price: tdPrice, value: tdValue, … };
  });
}

function updateLotRows(lots) {       // every change: write new text into the same cells
  lots.forEach(function (lot) {
    var r = lotRows[lot.id];
    setText(r.price, fmt.price(lot.price));
  });
}
```

`setText` itself checks `node.textContent !== text` before writing, so unchanged cells are not touched at all. Charts follow the same rule: `charts.instances[key]` holds each chart, and the second call updates its data instead of building a new one. Sections that are small and change rarely (advisories, the priority list) do rebuild with `innerHTML = ''` — that is a deliberate trade-off, not an oversight.

2.5 Publish and subscribe

Services do not know who is listening. Each keeps a list of functions and calls them when something happens:

```
var subscribers = [];

function publish(event) {
  subscribers.forEach(function (fn) {
    try { fn(event); } catch (err) { console.error('[market] subscriber failed', err); }
  });
}

subscribe: function (fn) {
  subscribers.push(fn);
  return function unsubscribe() { … };      // the caller can stop listening
}
```

`app.js` subscribes to `market`, `irrigation` and `pests` at boot, and every subscription ends in a call to `renderAll()` or `renderOperations()`. The `try/catch` inside `publish` matters: one listener throwing an error must not stop the others from being told.

2.6 Status objects

Every service exposes a `status` object and keeps it truthful: `state` is one of `idle | loading | ok | error`, with `lastError`, `lastFetched`, `latencyMs` alongside. The interface reads status; it never guesses. The "LIVE" pill in the top bar, the footer line "Open-Meteo · HTTP 200 · 412 ms", and the market source card are all just status objects rendered as text.

2.7 fetch, Promises and timeouts

Both APIs are called with `fetch`, which returns a **Promise** — an object that will have a value later. You chain `.then(…)` for the success path and `.catch(…)` for failure:

```
fetchWithTimeout(url, 12000)
  .then(function (res) {
    if (!res.ok) throw new Error('Open-Meteo returned HTTP ' + res.status);
    return res.json();                        // also a Promise
  })
  .then(function (raw) { cache = normalise(raw, location); … })
  .catch(function (err) {
    if (retriesLeft > 0) return attempt(retriesLeft - 1);   // one retry
    status.state = 'error'; status.lastError = err.message;
    throw err;                                 // let the caller know too
  });
```

`fetchWithTimeout` wraps `fetch` with an `AbortController` so a request that hangs is cancelled after 12 seconds instead of leaving the pill on "Loading" forever. Both services have their own copy of it.

2.8 Storage that cannot break the page

`store.js` wraps `localStorage` (survives closing the browser) and `sessionStorage` (survives page-to-page, cleared when the tab closes). Every call is inside `try/catch` with a fallback value, because storage can be full, blocked, or missing, and none of that may stop the dashboard rendering. Keys are prefixed `agrosense.` so they never collide with another site on the same host. Who uses it:

Key	Store	What it remembers
`location`	local	The last location picked in the top bar.
`prices`	local	The last successful WFP fetch, so the board paints instantly next time.
`weather`	session	The raw forecast, so moving between pages does not re-query Open-Meteo.
`irrigation`	local	Running valves, water applied today, the cost ledger — for today only.
`pests`	local	Treatments and scouting visits recorded in the interface.
`livestockDone`	local	Health-calendar tasks ticked off.
`harvestNotified`	local	Which harvest pings were sent today, so each fires once a day.
`toasted`	session	Which toasts have been shown this session.

2.9 Relative dates in the data

Nothing in `config.js` is a fixed date. Lots have `harvestedDaysAgo`, plantings `plantedDaysAgo`, health tasks `dueInDays`. Each is turned into a real date at run time (`new Date(now - days * DAY_MS)`). The reason is simple: a demo with fixed dates goes stale the day after it is written. With relative ages the spoilage maths, the harvest windows and the calendar are correct whenever the dashboard is opened.

2.10 The severity vocabulary

Every module that judges something uses the same words, and the CSS keys colours off them:

Level	Meaning	CSS hook
`critical`	Act now; money is being lost.	.badge.critical, [data-severity="critical"], [data-level="critical"]
`serious`	Act this week.	.badge.serious, …
`warning`	Watch closely.	.badge.warn / .warning
`ok / good`	Under control, or a good opportunity.	.badge.ok, .badge.good
`info / neutral`	Nothing to do.	.badge.neutral

Because the words are shared, a page can render an advisory from `weather.js`, a heat-stress row from `livestock.js` and a harvest alert from `harvest.js` with the same badge function and the same colours.

Part 3 — File by file

In load order. For each file: what it is for, the functions that matter, what it exposes, and what to try.

3.1 js/config.js — the data

EXPOSES NS.CONFIG, NS.SERIES, NS.LIVESTOCK_SERIES

The one file a farm would edit to make the dashboard theirs. It contains no logic — only objects and arrays — and it is loaded first so everything else can read it as `cfg`. If a number on screen is wrong and it is not a live price or the weather, it came from here.

Section	What it holds	Used by
`farm`	Name, manager, initials, hectares, year established.	shell.js (sidebar footer, avatar, greeting)
`locations`	Six Nigerian towns with lat/lon. The weather is fetched for whichever is selected.	app.js, weather.js, landing.js
`crops`	One entry per crop: WFP feed mapping, base price and volatility (for modelled prices), seasonality, shelf life, store, Kc, maturity window, harvest note, colour.	market, analytics, irrigation, harvest
`inventory`	Lots in store: crop, kg, age in days, cost per kg, grade, field.	analytics.js
`fields`	The six blocks and what is planted in each.	irrigation, pests, farm page
`plantings`	What is in the ground: field, crop, days since planting, expected t/ha.	harvest.js
`harvest`	Lead time for alerts, re-check interval, wet-day threshold.	harvest.js, app.js
`livestock`	Herds (head, live weight, dressing %, cost, age), products (mapped to WFP series), health calendar, feed cost.	livestock.js, market.js
`irrigation`	Water cost, depletion trigger, rain effectiveness, demo clock speed, the six zones with soil and root depth.	irrigation.js
`pests`	Spray limits, the watchlist (insects with base temp + generation DD; diseases with humidity/temp band), treatments on record.	pests.js
`market`	The HDX endpoint, months of history, minimum markets per month, cache hours, farmgate factor.	market.js
`weather`	Refresh interval, forecast days, past days, endpoint.	weather.js

> WHY IT IS BUILT THIS WAY
>
> **Colours are assigned per crop, once**
>
> `SERIES` is a list of six colours in a fixed order, validated so adjacent pairs stay distinguishable to colour-blind readers. Each crop is given its slot here and keeps it everywhere: the same blue is maize in the pie, the line chart, the sparkline and the table swatch. A crop must never be coloured by its *position* in a sorted list, because sorting would then repaint it. Livestock products have a second, muted list so they never collide with a crop on the same chart.

TRY THIS

Change the farm

Add a seventh lot to `inventory` (copy a line, change the id). Reload the cockpit. The hero value, the “Crops in store” tile, the sell priority and the lot table on the farm page all change — you did not touch any of them. That is what “config knows nothing, everything reads config” buys you.

3.2 js/dom.js — DOM helpers

EXPOSES NS.DOM

Function	Does	Example
`el(name)`	Find the element whose data-el is name.	el('tileMass')
`all(selector)`	querySelectorAll as a real array.	all('.nav-link')
`setText(node, text)`	Write text only if it changed; ignores a null node.	setText(el('x'), '42')
`tag(name, class, text)`	Create an element with a class and optional text.	tag('td', 'right num', '1.2 t')
`icon(id)`	An <svg><use href="#id"/></svg> from the sprite.	icon('i-harvest')
`badge(tone, icon, text)`	A with icon and label — the status pattern.	badge('critical', 'i-alert', 'Act now')
`flash(node, dir)`	Add tick-up / tick-down for 900 ms so a moved value glows.	flash(cell, 1)
`deltaClass(v)`	'up' / 'down' / 'flat' for colouring a change.	deltaClass(-0.02) → "down"
`mark(id, color)`	The product picture for a crop or livestock id, or the plain coloured square if it has none. Every page uses this one call.	mark(r.cropId, r.color)
`picture(id, color)`	A round product picture whose ring is the series colour — replaces the coloured square without losing the legend-to-line link.	picture('p-maize', '#2a78d6')
`peek(row, content, into)`	Hide extra text until the row is hovered, focused or clicked; makes the row focusable.	peek(li, p, body)
`has(name)`	Is there a data-el called name on this page?	if (has('heatList')) …

Notice `flash` contains `void node.offsetWidth;`. Reading `offsetWidth` forces the browser to apply the class removal before the class is added again — without it, two flashes in a row would merge into one. That line is a well-known trick, and it is the kind of thing worth a comment, which it has.

3.3 js/store.js — persistence

EXPOSES NS.STORE.LOCAL, NS.STORE.SESSION

Forty lines, but they protect every page. `make(storage)` returns an object with `get`, `set`, `remove`; each wraps the browser call in `try/catch` and JSON-encodes the value so you can store objects, not just strings. `safe(name)` checks whether `window.localStorage` can even be touched (some private windows throw on access). If it cannot, a `noop` object stands in — every `get` returns the fallback, every `set` returns false, and the dashboard carries on without memory.

3.4 js/format.js — formatting

EXPOSES NS.FMT

All number and date formatting goes through here so the whole site agrees on how a naira amount looks. It uses `Intl.NumberFormat`, the browser's built-in locale-aware formatter, configured once for `en-NG` and `NGN`.

Function	Input → output
`money(n)`	48215000 → ₦48,215,000 (no kobo)
`price(n)`	852.4 → ₦852.40 (kobo shown — for per-kg prices)
`moneyShort(n)`	48215000 → ₦48.2m; 12000 → ₦12k
`moneySigned(n)`	-3200 → -₦3,200; 3200 → +₦3,200
`mass(kg)`	24000 → 24 t; 640 → 640 kg
`pct(f), pctSigned(f)`	0.062 → 6.2%; → +6.2%
`head(n)`	2400 → 2,400 head
`dayMonth(d), monthYear(d), monthYearLong(d)`	→ 9 Sep; → Jun 26; → June 2026
`monthToDate("2026-06")`	→ the 15th of June 2026 (WFP dates its months by the 15th)
`inDays(n)`	0 → today; 1 → tomorrow; -3 → 3 days ago; 5 → in 5 days
`days(n)`	shelf life: 4 → 4d left; -2 → 2d over
`ago(ts)`	a timestamp → just now / 40s ago / 6m ago / 2h ago

Every function returns – for a missing or non-numeric value rather than printing `NaN`. That is why a page that has not loaded yet shows dashes rather than garbage.

3.5 js/icons.js — the icon sprite

EXPOSES NS.ICONS; INJECTS THE SPRITE INTO <BODY>

One SVG containing ~60 `<symbol>`s, each a 24×24 stroked drawing with an id like `i-harvest`. It is inserted at the top of `<body>` on load; from then on `<svg class="icon"><use href="#i-harvest"/></svg>` anywhere on the page draws that symbol. Because the strokes use `currentColor`, an icon takes the text colour of wherever it sits — which is why one sprite works on the dark sidebar and the white cards.

> TRY THIS
>
> **Add an icon**
>
> Copy any `<symbol>` line in the `SPRITE` string, give it a new id, and change the `d="…"` path (draw one at a 24×24 grid in any SVG tool, or borrow a path from an open icon set). Then use it with `icon('i-yourname')` in JS or `<use href="#i-yourname"/>` in HTML.

3.6 js/services/market.js — market prices

EXPOSES NS.MARKET

The longest service, because real data is awkward. Prices come from the **World Food Programme** market monitor, served through the **HDX Humanitarian API** — the one free, browser-callable feed of Nigerian food prices. Its shape dictates the module's shape:

- It is **monthly**, so the board shows the latest complete month and the change on the month before. There is no live tick.
- It is **retail**, at WFP-monitored markets (currently Borno, Yobe and Adamawa). `config.market.farmgateFactor` lets a farm apply its own spread.
- The newest month is usually **partial** — a few markets in early. A month counts once `minMarkets` (4) have reported.
- Not every crop has a series. Maize flour stands in for maize grain (a **proxy**); cassava and soybean have nothing and are **modelled**. The board labels each one.

Instruments

`instruments()` merges the crops and the livestock products into one list, each tagged `kind: "crop"` or `"livestock"`. From here on the module does not care which is which — an instrument is anything with a price.

Three tiers

```
init()
  │
  ├─ loadCache()     localStorage "prices"  ← the last live fetch
  ├─ loadSnapshot()  data/prices-snapshot.js ← bundled by tools/fetch-prices.py
  │      pick whichever is newer → apply() → publish('snapshot')      [page paints NOW]
  │
  └─ if not fresh (older than cacheHours): refresh()
         fetchCommodity() for every WFP commodity in parallel
         ↓ settle()  (Promise.allSettled for old browsers)
         merge with what we had, store.local.set('prices'), apply(), publish('update')
```

The order is the point: the page paints with real figures from the offline tier *before* any request is answered, and the live fetch quietly upgrades it. If the fetch fails, the status carries the error and the offline figures stay up.

From raw rows to a quote

Function	What it does
`compactSeries(rows, commodity)`	Groups raw API rows by month, keeps one price per market, takes the median across markets, converts to the base unit (2.1 KG → per kg). Returns {points:[{m, p, n}], unit, states…} where n is how many markets reported.
`latestCompleteMonth(series)`	The newest month where any series has n ≥ minMarkets. This becomes the end of the shared month axis.
`monthAxis(end, 24)`	The last 24 month keys, oldest first — every instrument is laid on this same axis so they can be summed.
`liveHistory(inst, compact)`	Maps the axis to closes. A month with no complete data carries the previous month forward and is marked filled:true — a flat segment rather than a hole.
`modelledHistory(inst)`	For instruments with no series: a mean-reverting random walk around basePrice × seasonalFactor, seeded deterministically so it is the same every reload.
`rebuild()`	Runs the above for every instrument, counts live/proxy/modelled, records which states reported, then calls rebuildQuote for each.
`deriveSignal(price, ma12, momentum)`	Sell now / Sell soon / Hold / Watch, by comparing this month with the 12-month average (see Part 5).
`rebuildQuote(id)`	Builds the quote object every page reads: price, prevClose, changePct, ma3, ma12, yoy, spark (last 12), signal, mode, a plain-English note on provenance.

WHY IT IS BUILT THIS WAY

Why fill forward instead of leaving a gap

The portfolio history multiplies every month's price by the kilos held. A `null` in one crop's series would make the whole farm value `null` for that month — a hole in the hero chart. Carrying the last known price forward and flagging it `filled: true` keeps the sum honest and lets the interface say "no new data this month".

TRY THIS

Watch the tiers

Open the live page and the console. `AgroSense.market.status.tier` says `"snapshot"`, `"cache"` or `"live"`. Clear the cache with `localStorage.removeItem('agrosense.prices')`, reload, and watch it go snapshot → live. Open the Network tab, filter "hapi", and you will see one request per commodity.

3.7 js/services/weather.js — weather intelligence

EXPOSES NS.WEATHER

Live data from **Open-Meteo** for the selected location. No API key, no sign-up. The request asks for far more than temperature: soil moisture and soil temperature, reference evapotranspiration (ET_0), precipitation probability, wind — because the irrigation, pest and livestock models all run off this one response. It also asks for `past_days=14`: the irrigation deficit is accumulated *backwards* from the last watering, and degree days are summed over the weeks a pest has been developing.

Function	What it does
`buildUrl(location)`	Assembles the query string from CURRENT_FIELDS, HOURLY_FIELDS, DAILY_FIELDS and the location.
`interpret(code)`	Turns a WMO weather code (0–99) into a label and an icon name.
`normalise(raw, location)`	The important one. Converts the API's parallel arrays into one tidy object: current, daily (from today, 7 days), pastDaily (the 14 days before), hourlyWindow (72 rows from now), soil, next48 (rain total, max probability, humid hours).
`deriveAdvisories(w)`	Seven rules that turn the forecast into things to do: heavy rain → hold fertiliser; dry + low soil moisture → irrigate; humid + warm → scout for disease; wind → do not spray; heat; a dry run → drying window. Each states the number it fired on.
`load(location)`	fetch with a 12 s timeout and one retry; on success normalise, cache in memory and sessionStorage.
`restore(location)`	On page load: if a forecast for this location is in sessionStorage and younger than refreshMs, re-normalise it instead of fetching.

WATCH OUT

Index 0 is not today

With `past_days` in the request, the daily arrays start two weeks back. `normalise` finds today by comparing ISO date strings, not by assuming index 0. Likewise the hourly arrays include hours already past, so `startIdx` is the first hour at or after now and every "next 48 hours" slice starts there. If you add a new hourly field, slice it through the same `slice()` helper.

3.8 js/services/irrigation.js — irrigation control

EXPOSES NS.IRRIGATION

Two jobs. The **water balance** answers "how thirsty is each zone?" with the FAO-56 method (Part 5.6). The **controller** lets valves be opened and closed, and a running valve genuinely changes the numbers.

Function	What it does
`accumulateDeficit(zone, crop, weather)`	Walks forward from the last watering day: each day add ETc (ET0 × Kc), subtract effective rain (× 0.75), floor at zero. Uses the real past days; falls back to a regional ET0 and flags estimated:true if the forecast is down. Subtracts water applied this session.
`rainRelief(weather, balance)`	Will the next three days' rain cover the deficit? Returns forecastMm, effectiveMm, covers, helps.
`recommend(zone, crop, balance, relief)`	The state: Rain-fed / Rain expected (only if a run was actually due) / Irrigate now (deficit ≥ RAW) / Due soon (≥ 75% of RAW) / Holding. Plus run time, volume and cost to close the gap.
`computeZones(weather), summarise(zones)`	One row per zone with all of the above; totals for the tiles.
`start(zoneId, minutes), stop(zoneId), stopAll(), startAllDue()`	The controls. start() refuses a rain-fed zone or one already running.
`advance(seconds)`	The clock. Every second, each open run gains seconds × simulationSpeed of run time; mm applied = rate × hours; volume = mm × ha × 10; cost = volume × ₦/m³. Finished runs close themselves.
`persist() / restore()`	The whole controller state lives in localStorage for the day. A run left open on one page is caught up on the time that passed when the next page opens.

> WHY IT IS BUILT THIS WAY
>
> **The demo clock is stated, not hidden**
>
> A 210-minute run at real speed would never be watched. `simulationSpeed: 60` makes one real second one minute of run time, and the zones card prints exactly that. The alternative — completing a run instantly — would make the water and cost figures meaningless. Honest acceleration beats a fake result.

3.9 js/services/pests.js — pest and disease control

EXPOSES NS.PESTS

Three models on the same forecast, and one rule that reaches back into the money.

Function	What it does
`degreeDays(pest, weather)`	Insects. Sums max(0, (Tmax+Tmin)/2 − baseTemp) over the past days since the last scouting visit, starting from ddAtLastScout, then projects the forecast forward. Returns the running series (for the chart), progress toward generationDD, and days to the crossing.
`infectionWindow(pest, weather)`	Diseases. The longest unbroken run of hours inside the humidity/temperature band in the next 48 h, against wetHoursTrigger. Progress is capped at 1 — "200% of an infection window" is not a thing.

Function	What it does
`sprayWindow(weather)`	The next run of ≥ 2 hours under maxWindKmh, under maxPrecipProb, under maxTempC. No window → the treatment button is disabled.
`treatmentRows(), getFieldConstraints()`	Treatments on record (config + session), each with days remaining on its pre-harvest interval. getFieldConstraints() returns {field: {daysRemaining, reason, detail, clearsOn}} for every block still inside a PHI.
`assess(weather)`	Everything above for every pest, ranked by level then progress, with the spray window, the treatment list and summary counts.
`treat(pestId), scout(pestId)`	The actions. treat() records a treatment on every field growing that crop and starts the PHI clock; scout() clears the overdue flag. Both persist and publish.

> WHY IT IS BUILT THIS WAY
>
> **getFieldConstraints() is a contract**
>
> It returns plain data — a field, a number of days, a reason. `analytics.js` and `harvest.js` read it without knowing what a pesticide is. That is how a spray recorded on the operations page can strand a cassava lot on the farm page and rewrite the sell priority on the cockpit: the constraint travels as data through `compute()`.

3.10 js/services/harvest.js — harvest alerts

EXPOSES NS.HARVEST

Every planting has a window: the crop's `maturityDays` [earliest, latest] counted from planting. `statusFor(daysToOpen, daysToClose)` places a block in one of five tiers — overdue, ready, imminent (≤ 7 days), approaching (≤ leadDays), growing. Then three joins decide whether "ready" means "today":

- `weatherWindow()` — is the opening day wet (≥ 8 mm)? Where is the first pair of dry days?
- `marketNote()` — is the price worth selling off the field, or should this crop go into store and wait?
- The PHI from `constraints` — a spray that clears *after* the window opens holds the harvest back.

`assess()` returns rows, alerts (anything not growing), livestock coming to market weight, and a summary. `notify()` sends browser notifications for critical/serious alerts, once a day per block, tracked in localStorage. `requestPermission()` is only called from a button — browsers ignore permission requests that a user did not trigger.

3.11 js/services/livestock.js — livestock

EXPOSES NS.LIVESTOCK

Function	What it does
`valueHerds(quotes)`	Head × live kg × dressing % = carcass kg, priced at the live meat price. Layers are held at book value (they are an income stream, not a carcass) and their eggs priced per day. Also feed cost per day and days to market weight.
`holdings()`	Carcass kg per product plus the layers' book value — what portfolioHistory needs to value the herd at each month's price.
`thi(temp, rh)`	Temperature-humidity index: 0.8T + (RH/100)(T − 14.4) + 46.4.
`heatStress(weather)`	For each herd, the peak THI in the next 48 h (air temperature for fish) against species-specific bands, hours above the mild threshold, and species-specific relief advice.

Function	What it does
`deriveAdvisories(weather)`	Moderate/severe heat stress as an advisory in the same shape weather.js uses, so the cockpit lists it alongside the field advisories.
`health(), markDone(id), reopen(id)`	The vaccination/treatment calendar with due-date levels; completion is stored in localStorage.

3.12 js/analytics.js — inventory analytics

EXPOSES NS.ANALYTICS

Where stock meets price. Everything the dashboard claims about money is computed here from three inputs: the lots in `config.inventory`, the quotes from `market`, and the clock.

Function	What it does
`spoilage(daysRemaining)`	A coarse flag: past window → 100% of value exposed; ≤ 2 days → 50%; ≤ 5 → 25%; ≤ 10 → 10%; else 0.
`valueLots(quotes, constraints)`	One enriched row per lot: market value (kg × price), cost value, margin, days in store, shelf life left, risk, value at risk — and, if the lot's field is under a PHI, blocked / blockedFor / stranded (blocked for longer than it will keep).
`byCrop(lots)`	Rolls lots up to one row per crop — what the pie and bar chart show. Keeps the worst risk and soonest expiry.
`summary(lots, crops)`	Totals: value, cost, margin, kg, value at risk, blocked value, sellable value, stranded value, top crop and its share.
`portfolioHistory(months)`	Today's holdings (crop kg + carcass kg + layer book value) valued at each of the last 24 months' prices. Holdings are held constant on purpose — see the box.
`sellPriority(crops)`	The ranking (Part 5.4). A PHI overrides both forces; a stranded lot goes to the top with "Will spoil before it can be sold".
`snapshot(quotes, constraints)`	Runs all of the above plus livestock.value() and merges the totals into farmValue / farmCost / farmMargin.

> WHY IT IS BUILT THIS WAY
>
> **The history chart holds the barn constant**
>
> The hero chart answers "what have prices done to the value of what I am holding?". If it also tracked when each lot arrived, a price fall would look like a sale and a harvest would look like a rally. Fixing the holdings isolates the price effect, which is the thing a farmer cannot see any other way. The label under the chart says exactly this.

3.13 js/charts.js — the chart layer

EXPOSES NS.CHARTS

All Chart.js configuration in one place. Two rules shape it. First, **charts are created once and updated in place** — each function checks `instances[key]` and, if the chart exists, swaps its data and calls `update('none')`. Second, **no chart has two y-axes**: where two measures have different scales (temperature and rainfall) they get two charts, because a dual axis invents a relationship the data does not contain.

Colours are read from the stylesheet at boot by `loadTokens()` (`getComputedStyle(document.documentElement).getPropertyValue('--chart-grid')`), so the palette has one

source of truth in `styles.css`.

Chart	Type	Where
`portfolioValue`	Single-series area line, 24 months, with an end-label plugin	Cockpit hero
`marketTrend`	One line per instrument, 12 or 24 months, legend toggles	Live page (crops; livestock)
`inventoryMix`	Doughnut of value by crop (≤ 6 segments), 2 px surface gaps	Farm page (crops; herds)
`valueVsCost`	Grouped horizontal bars: market value vs cost per crop	Farm page
`forecastTemp / forecastRain`	Two charts, one axis each: high/low lines; rainfall bars	Live page
`waterDeficit`	Grouped horizontal bars: deficit vs refill point per zone	Operations
`degreeDays`	Measured (solid) + forecast (dashed) accumulation vs a threshold line	Operations
`sparkline`	Raw canvas, no Chart.js — twelve of these redraw on every update	Live page tables

3.14 js/shell.js — the shared chrome

EXPOSES NS.SHELL

Each dashboard page carries four empty placeholders — `<div data-shell="sidebar">`, `topbar`, `board`, `footer`. `shell.build()` replaces each with real markup, so the sidebar is written once in JavaScript rather than four times in HTML. The `NAV` array at the top is the sidebar's content: four entries, one per page, each with an `href`, an icon and (optionally) the name of a count pill. `data-page` on `<body>` tells the shell which group is current.

Function	What it does
`buildSidebar / buildTopbar / buildBoard / buildFooter`	Create the chrome. The top bar includes the location select, the live pill, the alerts bell (and its hidden panel), refresh, and the avatar.
`bindDrawer()`	The hamburger, which does two jobs: on a narrow screen (≤ 860 px) it opens the sidebar as a drawer over the page; on a wide one it collapses the sidebar column and remembers the choice in localStorage (sidebarCollapsed). Escape closes the drawer; a nav tap closes it too.
`buildBoardCells / updateBoard`	The price rail: one cell per instrument, grouped crops / livestock, with a WFP / proxy / modelled tag and a flash on change.
`setCount(name, value, alert)`	The count pills. Hidden at zero — a "0" is not news. app.js sets one per page: cockpit (urgent advisories + harvest), operations (zones due or running + pests), farm (harvest alerts + heat + health).
`renderAlerts(assessment)`	The bell panel: harvest and livestock alerts, plus the notification-permission button.
`buildFooter()`	The page foot. Only the words "Data sources" show; the three provenance lines are always in the DOM (app.js keeps them current) and lift up in a dark card on hover or focus.
`toast(opts)`	A bottom-right toast that removes itself after 9 s.

Function	What it does
`enhanceInfo() / attachInfo()`	Moves every section description and every static chart caption behind a small “i” beside its heading, served by ONE shared popover. Hover or keyboard focus opens it, a click pins it (the touch route), Escape or an outside click closes it. Captions with a data-el carry live figures and are left visible.

3.15 js/app.js — the orchestrator

EXPOSES NS.APP

The file that knows *when*. It has no drawing code of its own.

Function	What it does
`compute()`	Asks every service for its answer and packs one ctx object: quotes, weather, snapshot, history, advisories (weather + livestock, sorted by severity), irrigation, pests, harvest, heat, health, constraints.
`chrome(ctx)`	Updates the board, every sidebar count, the alerts panel, the sources footer and the live pill.
`renderAll()`	compute → chrome → pageModule.render(ctx). The heavy path.
`renderOperations()`	compute → the two operations counts → pageModule.renderOperations(ctx). The light path, used by the irrigation clock so the whole page is not rebuilt every second.
`loadWeather()`	weather.load → renderAll → pingHarvest, whether or not it succeeded (the error is kept in state).
`pingHarvest()`	Browser notifications via harvest.notify, and in-app toasts once per session for urgent items.
`bindChrome()`	The location select (saved to localStorage) and the refresh button (weather + prices).
`boot()`	The startup sequence — see Part 4.1.

> WHY IT IS BUILT THIS WAY
>
> **Two ways of tucking text away, for two kinds of text**
>
> **Explanation** — what a section is for, how to read a chart — sits behind the “i” (`attachInfo`). It is the same for every visitor and every refresh, so it is lifted out of the HTML once, at load.
>
> **Reasoning about one row** — why cassava is ranked second, how far a block is through its window — opens when that row is hovered (`dom.peek`). It is generated per row from live data, so it is built where the row is built.
>
> The reveal animates `grid-template-rows` from `0fr` to `1fr` — the one CSS trick that animates to an height you do not know in advance. Both patterns keep the text in the DOM, so screen readers still reach it.

3.16 js/pages/*.js — the page modules

Each registers `NS.pages.<name>` with up to three functions. `init()` runs once before any data: it builds static rows and binds buttons. `render(ctx)` runs on every change and only reads `ctx`. `renderOperations(ctx)` (operations page only) is the cheap per-second path.

Module	init()	render(ctx) fills
`cockpit.js`	Starts the clock in the conditions card.	The whole cockpit grid — conditions and the seven days ahead, six tiles, the market feed, what the farm is worth, today's jobs — then the sell priority, advisories and harvest calendar below it.
`live.js`	Builds the crop and livestock table rows and legends; binds the 12/24-month range buttons.	The seven-day temperature and rainfall charts, both market tables and trend charts, the provenance card.
`operations.js`	Binds "Open all due zones" / "Close all"; the pest select.	Irrigation tiles, deficit chart, zone cards with valve buttons and progress bars; pest tiles, degree-day chart, treatments, watchlist with Record treatment / Log scouting.
`farm.js`	Binds the sortable column headers.	Lot table (built once, updated in place) + footer, mix and value-vs-cost charts, field tiles, harvest calendar with window timelines, herd table, heat stress, health calendar.

TRY THIS

Follow one number from data to screen

Pick the "Livestock on the hoof" tile. In `cockpit.html` find `data-el="tileStock"`. In `pages/cockpit.js` find `setText(el('tileStock'), …)` — it reads `ctx.snapshot.livestock.summary.totalValue`. In `app.js`, `compute()` got `snapshot` from `analytics.snapshot()`, which called `NS.livestock.value(quotes)`. In `livestock.js`, `valueHerds` multiplied head × kg × dressing × price. The price came from `market.js`, the head count from `config.js`. Five files, one direction.

3.17 js/landing.js and index.html — the landing page

Deliberately standalone: it loads only `config.js`, the price snapshot and itself, so it is fast and cannot break if a dashboard file does. It makes one small Open-Meteo request for the default location, reads one clean series (rice) from the bundled snapshot, and finds the soonest harvest from `config.plantings`. If the weather request fails the chips say so rather than showing stale numbers. "Log in" is a link.

3.18 css/styles.css — the dashboard stylesheet

About 1,600 lines, organised top to bottom in the order things appear on screen. The section banners (`/* === Shell === */`) are the map — search for them.

Tokens first

Lines 22–120 define every colour, radius and font as a **CSS custom property** on `:root`: `--canopy-600: #28a95e;`. Everywhere else says `var(--canopy-600)`. Change the token and every use changes; this is also how `charts.js` gets its colours — it reads them at boot with `getComputedStyle`. That is why the whole interface could be re-skinned without touching a single chart file.

The tonal ladder — the one idea the look is built on

There are no borders and no shadows anywhere. Depth is carried by **tone**, and the ladder runs the opposite way to most interfaces: surfaces get *lighter* as you go deeper.

```
--backdrop      #d8d9db    the desk the app panel sits on
--shell         #e3e3e3    the app panel itself
--surface       #f2f2f2    a card on the panel
```

```
--surface-sunk  #f9f9f9    a row INSIDE a card — lighter, not darker
--surface-lift  #ffffff    the brightest step: inputs, hover, chips
```

So a card reads as a card because it is a step lighter than the panel, and a row reads as nested because it is a step lighter again. Add a border and the effect collapses — which is why `--hairline` is almost invisible and every card sets `border: 0` explicitly.

On top of that sit exactly three accents: one saturated green (`--canopy-600: #28a95e`) for the single headline tile, the weather card and every chart line; near-black (`--soil-900: #121212`) for the price board, the active rail chip and count badges; and the reserved status four (good / warning / serious / critical) which never carry anything but state. Everything else on the screen is grey.

The shell, for reference

Lines	What
172–180	.shell — the rounded panel: a flex row, full height, overflow hidden
182–193	.sidebar — the 84px icon rail and its width transition
195–231	.brand, .brand-mark, .brand-text — the green disc; the wordmark is drawer-only
242–262	.nav-link — the 52×52 chip, hover, and .is-active (black)
264–281	.nav-count — the badge on the chip corner, with a shell-coloured ring
282–309	.sidebar-foot and .nav-logout
311–323	.main and the desktop collapse (body.sidebar-collapsed sets the rail to width 0)
325–345	.topbar — a fixed header row, no longer sticky
433–470	.board — the black price rail
518–535	.content — the ONLY scrolling element on the page
548–560	.card — a lighter step, radius 20, no border, no shadow
650–686	.tile and .tile-accent — the stat tiles and the one green headline tile
1586– (@media 860px)	Where the rail becomes a labelled drawer

The app-shell layout

The page itself does not scroll. `body` is `height: 100dvh; overflow: hidden` with a little padding, the `.shell` panel fills it, and inside the panel only `.content` scrolls. That is what keeps the top bar and the price board permanently in view and gives the app its "one panel" feel.

```
body        100dvh, overflow hidden, padding: var(--inset)   ← the backdrop
  .shell    height 100%, radius 26px, overflow hidden, flex row
    .sidebar   84px rail, flex: none
    .main      flex column, overflow hidden
      .topbar  flex: none
      .board   flex: none
      .content flex: 1, overflow-y: auto      ← the scroller
```

One consequence worth knowing: `window.scrollTo` and `document.documentElement.scrollTop` do nothing now. If you need to scroll the page from JavaScript, scroll `document.querySelector('.content')` instead.

`scrollIntoView()` still works, because it finds the right scroll container on its own.

Specificity in one example

```
.nav-link .nav-count          { background: rgba(255,255,255,0.12); }   /* two classes   */
.nav-link .nav-count.is-alert { background: var(--critical); }          /* three classes */
```

Both rules match an alert pill. The second wins because it is *more specific* (three class names beat two), not because it comes later — although later does break ties. When a style "does not apply", this is the first thing to check; DevTools strikes through the losing property.

Responsive

Four breakpoints near the bottom: `1180px` (tiles 4→2), `980px` (two-column grids → one), `860px` (the sidebar becomes a drawer), `620px` (tiles → 1, controls stack). A `prefers-reduced-motion` block turns off the entrance animation for people who have asked their OS for that.

A BUG WORTH REMEMBERING

overflow: hidden does not clip position: absolute

The panel clips its contents with `overflow: hidden`, yet jumping to `live.html#market` once scrolled the *whole window* and left the panel cut off. The cause: an absolutely-positioned element is only clipped by an ancestor that is itself **positioned**. The visually-hidden text spans (`.sr-only`, which the info buttons add all the way down the page) were absolute with no positioned ancestor, so they escaped the panel and stretched the document to 2,056 px against an 808 px window.

The fix was two lines: `position: relative` on `.shell` and `.content`. Test for it with `document.documentElement.scrollHeight` — in an app-shell layout it must equal `innerHeight`, and any bigger number means something has escaped.

WATCH OUT

The ladder is only four steps wide — do not invent a fifth

Every surface in the interface is one of `--shell`, `--surface`, `--surface-sunk` or `--surface-lift`, and they sit within about twenty levels of each other. Drop in a hand-picked grey like `#eaeaea` and it will read as a mistake rather than a layer, because it lands between two steps.

The same restraint applies to the accent. One green tile per screen is the rule; two and neither reads as the headline. The green tile on the cockpit is marked with the class `tile-accent` in `cockpit.html` — move the class to change which number leads.

3.19 css/landing.css

A separate, smaller sheet for `index.html` only: the full-bleed photograph and its shade, the grid overlay, the telemetry chips, and the entrance animation. It shares the token names with `styles.css` so the two look like one product.

3.20 The HTML pages

Every dashboard page has the same skeleton, and only the middle differs:

```
<body data-page="cockpit">                    ← which nav group is current
  <div class="shell">
    <div data-shell="sidebar"></div>          ← shell.js replaces these four
    <div class="main" id="main">
      <div data-shell="topbar"></div>
```

```
      <div data-shell="board"></div>
      <div class="content">
        <section class="section" id="overview">   ← id matches the nav hash
          … cards with data-el placeholders …
        </section>
      </div>
      <div data-shell="footer"></div>
    </div>
  </div>
  <script src="…Chart.js…"></script>
  <script src="js/config.js"></script>          ← then the rest, in dependency order
  …
  <script src="js/pages/cockpit.js"></script>
  <script src="js/app.js"></script>             ← always last
</body>
```

Section `id`s are the sidebar's anchors, so renaming one breaks a link. `aria-labelledby`, `.sr-only` headings and `<caption>`s on tables are there for screen readers; keep them when you copy a block.

3.21 tools/fetch-prices.py and data/prices-snapshot.js

The snapshot is the offline tier for prices. The Python script calls the same HDX endpoint the browser does, does the same median-across-markets reduction, and writes a compact JavaScript file. Run it whenever you want fresher bundled data:

```
python tools/fetch-prices.py       # writes data/prices-snapshot.js; no packages needed
```

The generated file must not be edited by hand — it says so at the top. The browser will overwrite its own cached copy anyway on the next successful fetch.

3.22 js/services/tasks.js — today's jobs

EXPOSES NS.TASKS

The task list on the cockpit is not a separate to-do app. It is the rest of the dashboard collapsed into one ranked list of things a person could go and do this morning, and every row carries the reason the service that produced it gave:

Row	Comes from	Fires when
Harvest a block	harvest.js	the block is inside or past its window and the level is critical or serious
Irrigate a zone	irrigation.js	the deficit has passed the refill point and the zone has a valve
Treat a pest	pests.js	the threshold is crossed AND a spray window is open
Scout a block	pests.js	not walked in over ten days — unless that pest is already queued for treatment, since spraying means visiting
Vaccinate a herd	livestock.js	the health calendar has it due inside three days
Move the stock	analytics.js	the top of the sell priority scores 0.55 or more

Ticking a row stores the completion for the rest of the day under `agrosense.tasksDone`, keyed by date so yesterday's ticks do not carry over. The one exception is a livestock health row, which calls `livestock.markDone()` instead — the farm page owns those, and the two views have to agree.

WHY IT IS BUILT THIS WAY

Why the list is derived rather than stored

Nothing is ever added to this list by hand. If a task is on it, some model put it there and can be argued with; if the weather changes and a spray window closes, the "treat" row disappears on the next render. A to-do list you maintain separately goes stale the first busy week. One that is recomputed from the forecast and the barn cannot.

Part 4 — Walkthroughs

Trace these with the console open. Each step names the function; put a `console.log` in it if you want to see it fire.

4.1 Opening cockpit.html

1. The browser parses the HTML. The four `data-shell` placeholders and every `data-el` element exist but are empty.

2. Scripts load in order. `config.js` creates `window.AgroSense`. Each following file attaches its part. `icons.js` injects the sprite. `pests.js` and `irrigation.js` restore their localStorage state as they load.

3. `app.js` runs `boot()` when the DOM is ready.

4. `boot` reads the saved location (or the default), then `charts.init()` reads the CSS tokens.

5. `shell.build()` replaces the placeholders with the sidebar, top bar, board and footer; binds the drawer and the bell; builds the board cells (still showing "—").

6. `bindChrome()` fills the location select and wires the refresh button.

7. The page module's `init()` runs (the cockpit has none; the farm page builds its table rows here).

8. `app` subscribes to `market`, then calls `market.init()`. That loads the cache or snapshot and publishes **synchronously** — so the subscriber fires immediately and `renderAll()` paints real prices before any network request exists.

9. Inside `renderAll`: `compute()` builds `ctx` (with `weather: null` for now), `chrome(ctx)` fills the board and counts, `pages.cockpit.render(ctx)` fills the hero and tiles. Weather-dependent sections say "Waiting on the forecast…".

10. If the price cache is stale, `market.refresh()` starts in the background.

11. `app` subscribes to `irrigation` and starts its one-second clock; subscribes to `pests`.

12. `weather.restore()` is tried. On the first page of a session it returns null, so `loadWeather()` runs: the pill goes to "Loading", `weather.load()` fetches Open-Meteo.

13. The forecast arrives (~400 ms). `normalise` shapes it, it is cached in sessionStorage, `renderAll()` runs again — now advisories, irrigation, pests, harvest and heat stress all have data — and `pingHarvest()` shows toasts for anything urgent.

14. Some time later the WFP fetch returns; `market` publishes `update`; `renderAll()` runs a third time and the board cells flash where a price moved.

15. From here: the irrigation clock ticks every second (only doing work if a valve is open); weather re-checks every minute and re-fetches after 15 minutes; the harvest check re-runs hourly; the tab going hidden pauses the clock.

4.2 Clicking "Record treatment" on cassava mealybug

1. In `pages/operations.js` the button's click handler opens a `confirm()` dialog naming the product, the fields and the PHI.

2. On OK it calls `NS.pests.treat('mealybug')`. That finds every field growing cassava (Field C), pushes a treatment record with `appliedAt: Date.now()` and `phiDays: 3` into `sessionTreatments`, persists to localStorage, and publishes.

3. `app.js`'s pest subscription calls `renderAll()`.

4. `compute()` calls `NS.pests.getFieldConstraints()`, which now includes `{ "Field C": { daysRemaining: 3, … } }`.

5. `analytics.snapshot(quotes, constraints)` runs `valueLots`. Both cassava lots are from Field C, so both get `blocked: true`. LOT-2427 has 1 day of shelf life left and is blocked for 3, so `stranded: true` and its whole value becomes `strandedValue`.

6. `byCrop` rolls that up; `sellPriority` sees `row.strandedValue > 0` and returns the "Will spoil before it can be sold" item with score 1.2 — above everything else.

7. `harvest.assess` also receives the constraints: the cassava planting in Field C gains a `phi` note if the interval clears after its window opens.

8. Every page then redraws from the new `ctx`: the cockpit's sell priority puts cassava first with a "Sale blocked" badge; the farm page's lot table shades the two rows; the operations page lists the new treatment with "3d to clear"; the sidebar counts update.

9. Reload the page: `pests.js` restores the treatment from localStorage on load, so the block survives. To undo it, clear the key: `localStorage.removeItem('agrosense.pests')`.

4.3 Changing the location

1. The `<select>` fires `change`. `bindChrome`'s handler finds the location object, stores its id in localStorage, and calls `loadWeather()`.

2. `weather.load(location)` fetches Open-Meteo for the new coordinates. The session cache is replaced.

3. `renderAll()` → `compute()`. Nothing about prices or stock changed, but every forecast-driven model re-runs: advisories, the irrigation deficits (a drier town → larger deficits), infection windows, the spray window, heat stress, the harvest weather join.

4. Try Ibadan then Kano and watch the irrigation zones go from "Holding" to "Due soon", and rice blast from critical to clear.

4.4 Opening a valve

1. "Open valve" calls `NS.irrigation.start(zoneId, minutes)` with the minutes `recommend()` computed to close the deficit. A run object is created and persisted; `publish('started')` fires.

2. `app.js`'s irrigation subscription calls `renderOperations()` — the light path — which redraws the zone cards and the two sidebar counts only.

3. Every second `loop()` → `advance(1)`: the run gains 60 minutes of run time, mm applied = rate × 1 h, the ledger grows, `persist()`, `publish('progress')` → the progress bar moves.

4. When `elapsedMinutes` reaches the plan, the run is deleted and `publish('completed')` fires. Next `computeZones` subtracts the applied mm from the deficit, so the zone reads "Holding".

5. Navigate to another page mid-run: `restore()` on the next page reads the saved run, sees how many real seconds passed, and calls `advance(gap)` to catch up.

Part 5 — The maths, with worked examples

Every number on screen comes from one of these. The examples use the figures in `config.js` and a plausible price, so you can check them against the dashboard.

5.1 What a lot is worth

```
marketValue = quantityKg × price          LOT-2411: 24,000 kg × ₦880.95 = ₦21,142,800
costValue   = quantityKg × costPerKg      24,000 × ₦610 = ₦14,640,000
margin      = marketValue − costValue     ₦6,502,800   (+44% over cost)
```

The farm value on the cockpit is the sum of every lot's market value plus the livestock value (5.9). "Margin over cost" is the same sum against the same lots' cost.

5.2 Spoilage and value at risk

```
daysRemaining = shelfLifeDays − harvestedDaysAgo
LOT-2427 cassava: 7 − 6 = 1 day left   → spoilage factor 0.50 (≤ 2 days: "Move immediately")
valueAtRisk   = marketValue × factor   → 22,400 kg × ₦562.71 × 0.50 = ₦6,302,352
```

The tile "Value at spoilage risk" sums `valueAtRisk` over every lot. It is deliberately coarse — a farmer wants a flag, not a decimal.

5.3 The price signal

```
ratio = thisMonth / average of last 12 months
ratio ≥ 1.10 and rose last month  → Sell now
ratio ≥ 1.04                      → Sell soon
ratio ≤ 0.94                      → Hold  (selling here locks in the dip)
otherwise                         → Watch
```

It is a *position*, not a prediction: is this month strong or weak against the crop's own recent history?

5.4 Sell priority

```
spoilUrgency  = clamp(1 − soonestExpiry / 30, 0, 1)     30+ days out → 0; today → 1
marketUrgency = { sell: 1, soon: 0.7, watch: 0.4, hold: 0.1 }[signal]
score         = 0.55 × spoilUrgency + 0.45 × marketUrgency

Cassava: 1 day left → spoil 0.97; signal "hold" → 0.1     score = 0.58
Maize:  206 days left → spoil 0;   signal "soon" → 0.7    score = 0.32
```

Spoilage is weighted slightly higher because a rotten tuber has no price at all. A pre-harvest interval overrides the score entirely: blocked → score × 0.1; blocked *and* stranded → 1.2, above every ordinary item.

5.5 Farm value over time

```
for each of the last 24 months m:
  value[m] = Σ (kg held of crop c today × price[c][m])        crops in store
           + Σ (carcass kg of product p × price[p][m])       herds
           + layers' book value                              constant
```

Holdings are today's, held constant; only prices move. So the curve is "what would my current barn and herd have fetched each month?" — the price effect alone.

5.6 Irrigation: the FAO-56 water balance

```
TAW     = awcMmPerM × rootDepthM              ZN-A: 120 × 0.60 = 72 mm   (soil's usable water)
RAW     = TAW × depletionTrigger              72 × 0.5 = 36 mm            (what the crop draws comfortably)
ETc     = ET0 × Kc                            4.2 × 1.20 = 5.0 mm/day     (maize's daily use)
each day since the last watering:
  deficit = max(0, deficit + ETc − rain × 0.75)

5 dry days:            deficit = 5 × 5.0 = 25 mm  → 69% of RAW → "Due soon"
5 days with 6 mm rain: 5 × (5.0 − 4.5) = 2.5 mm    → "Holding"
to close a 25 mm deficit on 26 ha at 8 mm/h:
  run time = 25 / 8 = 3.1 h;  volume = 25 × 26 × 10 = 6,500 m³;  cost = 6,500 × ₦165 = ₦1.07m
```

ET_0 and rainfall are the real past days from Open-Meteo. That is why the same zones read “Holding” at Ibadan in the rainy season and “Due soon” at Kano.

5.7 Pests: growing degree days

```
GDD(day) = max(0, (Tmax + Tmin) / 2 − baseTemp)
Yam beetle: base 14 °C, generation at 450 DD, 210 DD at the last scout 15 days ago
a 30/23 °C day: (30 + 23)/2 − 14 = 12.5 DD
15 such days: 210 + 15 × 12.5 = 397.5 DD  → 88% → "Treat this week"
days to crossing = (450 − 397.5) / 12.5 ≈ 4
```

Development runs on warmth, not the calendar — which is why the model needs the past days and why a warm week brings the next generation forward.

5.8 Diseases: infection windows

```
count the LONGEST UNBROKEN run of hours in the next 48 where
  humidity ≥ minRh  and  tempRange[0] ≤ temp ≤ tempRange[1]
Tomato late blight: ≥ 85% RH at 10–26 °C, trigger 8 hours
16 consecutive humid hours → window met → critical.  Progress is capped at 1.
```

Scattered humid hours do not infect a crop; consecutive ones do. Hence the longest run, not the total.

5.9 Livestock: carcass value and eggs

```
carcassKg   = head × avgLiveKg × dressing       cattle: 42 × 310 × 0.52 = 6,770 kg
marketValue = carcassKg × meat price            6,770 × ₦6,800 = ₦46.0m
layers:      marketValue = head × costPerHead (book)   2,400 × ₦4,200 = ₦10.1m
             eggs/day = head × layRate = 2,400 × 0.82 = 1,968; income = 1,968 × ₦220 = ₦433k/day
feed/day    = head × feedCostPerHeadDay[species]
```

5.10 Heat stress: the temperature–humidity index

```
THI = 0.8 × T + (RH / 100) × (T − 14.4) + 46.4
32 °C at 70% RH: 25.6 + 0.7 × 17.6 + 46.4 = 84.3
cattle bands: mild 68, moderate 72, severe 80  → severe
goat bands:   mild 74, moderate 80, severe 86  → moderate (a Dwarf goat minds it less)
```

Part 6 — Glossary

JavaScript and browser terms as used here

Term	Meaning
`IIFE`	Immediately-invoked function expression: (function(){ … })(); — a private scope that runs once.
`Namespace`	The one shared object, window.AgroSense, that every file attaches to.
`DOM`	The browser's live tree of elements. JavaScript changes the page by changing the DOM.
`data-* attribute`	A custom HTML attribute (data-el, data-shell, data-page). Read in JS via dataset or querySelector.
`Promise / .then / .catch`	A value that arrives later; chain what to do on success and failure.
`fetch`	The browser's HTTP request function; returns a Promise of a Response.
`AbortController`	Lets a fetch be cancelled — used for the 12 s / 45 s timeouts.
`localStorage / sessionStorage`	Per-site key–value storage; local survives restarts, session lasts one tab.
`Notification`	The browser notification API — harvest pings, only after the user grants permission.
`CSS custom property`	A variable in CSS: --canopy-800, read with var(--canopy-800).
`Specificity`	Which CSS rule wins when several match: more ids > more classes > more elements; ties go to the later rule.
`Media query`	@media (max-width: 860px) { … } — rules that apply only at some screen sizes.
`Canvas`	A bitmap element you draw on with JavaScript; Chart.js and the sparklines use it.
`pub/sub`	Publish–subscribe: a service calls every registered listener when something happens.

Farming terms

Term	Meaning
Farmgate price	What the farmer receives at the farm, before transport and market margins. WFP publishes retail; farmgateFactor adjusts.
Lot	A batch of harvested produce in store, tracked as one unit.
Shelf life	How long a crop holds condition in its store — 7 days for fresh cassava, 300 for soybean in a silo.
Pre-harvest interval (PHI)	Days after applying a pesticide before the crop may legally be harvested/sold.
ET0 / ETc	Reference evapotranspiration (what a standard grass surface loses to the air) and crop evapotranspiration (ET0 × the crop coefficient Kc).
TAW / RAW	Total and readily available water in the root zone — FAO-56 terms.
Growing degree days	Accumulated warmth above a base temperature; insects develop on it.
Dressing percentage	Carcass weight as a share of live weight — 52% for cattle, 72% for broilers.
THI	Temperature–humidity index, the standard measure of heat stress in livestock.
Scouting	Walking a field to look for pests and disease; the models count days since the last visit.

Part 7 — A study plan for two

Five sessions of about two hours. Alternate who drives; the other reads this guide aloud and asks “why?”.

Session	Read	Do
1 · The map	Parts 1 and 2. Fix the rename bug first.	Run the site. In the console, explore AgroSense. Find one data-el in HTML and its setText in JS. Change a config number and watch it propagate.
2 · Data in	3.6 market, 3.7 weather, 4.3.	Network tab: find the HDX and Open-Meteo requests, read the JSON. Break the weather URL on purpose (edit endpoint) and watch the error state. Put it back.
3 · The joins	3.12 analytics, 3.9 pests, 3.10 harvest, 4.2, 5.1–5.5.	Record a treatment and follow the chain across three pages. Check one lot’s value by hand against the table. Change a shelf life and see the priority reorder.
4 · Operations & livestock	3.8, 3.11, 4.4, 5.6–5.10.	Open a valve; switch location to Kano; verify the deficit maths for one zone by hand. Tick a health task; find the localStorage key it wrote.
5 · Drawing	3.13 charts, 3.14 shell, 3.16 pages, 3.18 CSS.	Add a sixth tile to the cockpit (HTML + one setText). Change the sidebar palette. Add an icon to the sprite and use it. Make .tile:hover lift.

Exercises, easiest first

- **Rename a nav item.** In `shell.js`, change “The cockpit” to “Overview” in the NAV array. One string.
- **Add a location.** Add your town to `config.locations` with its coordinates. It appears in the select.
- **Add a crop.** A seventh crop needs a colour (extend `SERIES` — but note the pie’s six-segment guidance), a config entry, at least one lot, a field, a planting and a pest. Watch what breaks when you leave one out.
- **A new advisory rule.** In `weather.deriveAdvisories`, add “frost risk” when `tempMin < 5`. Give it a severity, an icon and a window.
- **A new tile.** “Feed cost per day” on the cockpit from `ctx.snapshot.livestock.summary.feedPerDay`.
- **Sort the market table.** Copy the `bindSorting` pattern from `farm.js` to `live.js`.
- **A dark theme.** Add a second set of token values under `[data-theme="dark"]` and a toggle that sets the attribute and remembers it in `store.local`. Then fix the charts: `charts.init()` reads tokens once, so it needs re-running.
- **Replace the modelled prices.** Find a second price source for cassava and add it as a fourth tier in `market.js`. Keep the provenance honest.

Appendix A — Where to change what

I want to…	Edit	Notes
Change the brand name	shell.js line 68 (sidebar); <title> in each HTML; index.html	Never the last line of a JS file.
Change the farm name / hectares	config.farm	Shows in the sidebar footer and greeting.
Change a price's source	config.crops[x].feed / livestock.products[x].feed	null = modelled; add proxy text if a related series stands in.
Change the panel or card tone	styles.css :root — --shell, --surface, --surface-sunk	Keep the steps evenly spaced or the layering stops reading.
Change the count pills	styles.css 264–281	Black by default; the ring is what separates them from the active chip.
Change the top bar	styles.css 325–345	It is a flex row inside the panel, not a sticky bar.
Add a page	a new HTML file + a page module + a NAV entry in shell.js	data-page on <body> must match the NAV id.
Add a stat tile	HTML .tile with two data-el names + setText lines in the page module	See 3.16 "Follow one number".
Change the demo irrigation speed	config.irrigation.simulationSpeed	The zones card prints the figure.
Change how often weather refreshes	config.weather.refreshMs	Also the sessionStorage freshness window.
Refresh the bundled prices	python tools/fetch-prices.py	Do not edit the output file by hand.
Add an icon	icons.js SPRITE	Then icon("i-name") or <use href="#i-name">.
Change a chart's colours	styles.css tokens (--series-*, --chart-grid, --canopy-600)	charts.js reads them at boot with getComputedStyle.

Appendix B — Debugging checklist

- **A page is blank or stuck on "—".** Console first. A red *SyntaxError* means a file did not parse — nothing after it ran. `node --check <file>` finds it without a browser.
- **One tile is stuck on "—".** A `data-el` name differs between HTML and JS. Search both.
- **A style does not apply.** Elements tab → select the element → Styles. The winning rule is on top; losers are struck through. Check specificity (3.18) and that the media query matches your width.
- **The weather is "offline".** Network tab → filter "open-meteo" → click the request → Response. HTTP 400 means a bad parameter; failed means no internet or a blocked host.
- **Prices are all "modelled".** `AgroSense.market.status` in the console: `tier`, `lastError`. If HDX is down the snapshot should still load — check `data/prices-snapshot.js` is in the script list.
- **Something changed and will not go back.** It is in storage: `Object.keys(localStorage)` lists the `agrosense.*` keys; `localStorage.clear()` resets the lot.

- **A number looks wrong.** Find the formula in Part 5, compute it by hand from `config.js` and the quote, and compare. Then put a `console.log` at the line that computes it.
- **The drawer will not open.** Only dashboard pages have it, and only under 860 px wide. Check the console for an error earlier in `shell.build()`.